



Java Database Programming, Testing & Documentation

[Visit our website](#)

Introduction

In this lesson, we learn how to connect to a MariaDB database with JDBC, and cover procedures for testing and documentation. At the end, there is a task for you to put into practice what you have learned.

Introduction to Database Connectivity

JDBC is the Java API for developing Java database applications. Liang (2011, p. 1286) defines JDBC as providing a uniform interface to Java programmers for accessing and manipulating a wide range of relational databases. “Using the JDBC API, applications written in the Java programming language can execute SQL statements, retrieve results, present data in a user-friendly interface, and propagate changes back to the database” (pp. 1286-1287). This makes data handling an easy and user-friendly process. To use JDBC, you need to download a JDBC driver to link to the database you want to work with. Each database has its own specific JDBC driver, such as the MariaDB JDBC driver to access a MariaDB database. MariaDB is a drop-in replacement for MySQL and is fully compatible for the purposes of this course.

In summary with the JDBC API, you are able to (Ciubotaru, & Muntean, 2013, p. 136):

1. Establish a connection with a database or access any tabular data source
2. Send SQL statements
3. Process the results

Setting up your Environment

Before you start developing with JDBC, you need to set up your environment. This includes downloading and installing **MariaDB**, as well as the JDK and a text editor. At this point, you should have already installed the JDK and a text editor such as TextPad, Notepad++ or Sublime. However, if not, you can download the [JDK here](#) and the [text editor Sublime here](#).

We will now explain the steps to download and install **MariaDB**.

Download and Install MariaDB

For Windows:

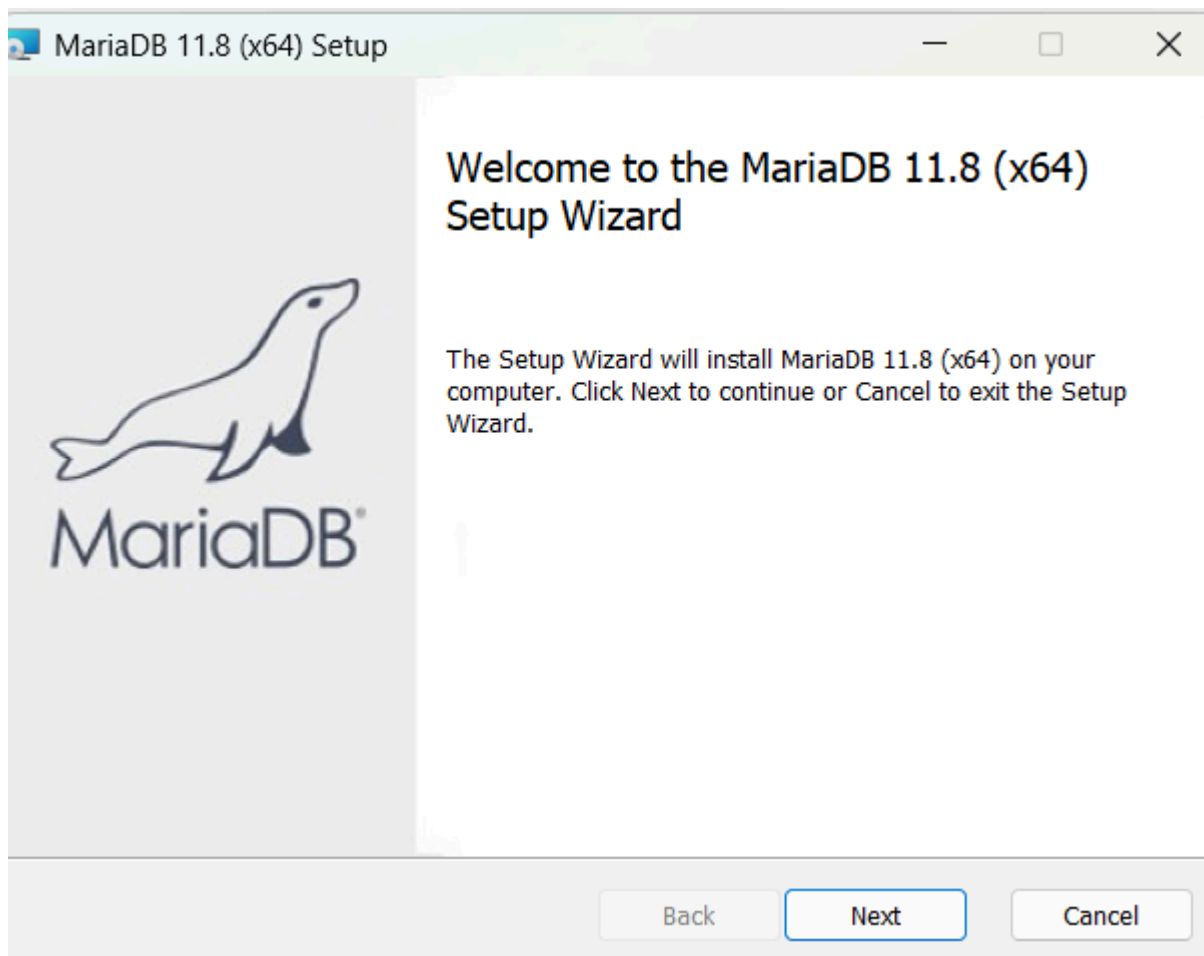
1. [Download MariaDB Server - MariaDB.org](#) installer from the official site and execute it.
2. Select Microsoft Windows as your OS.

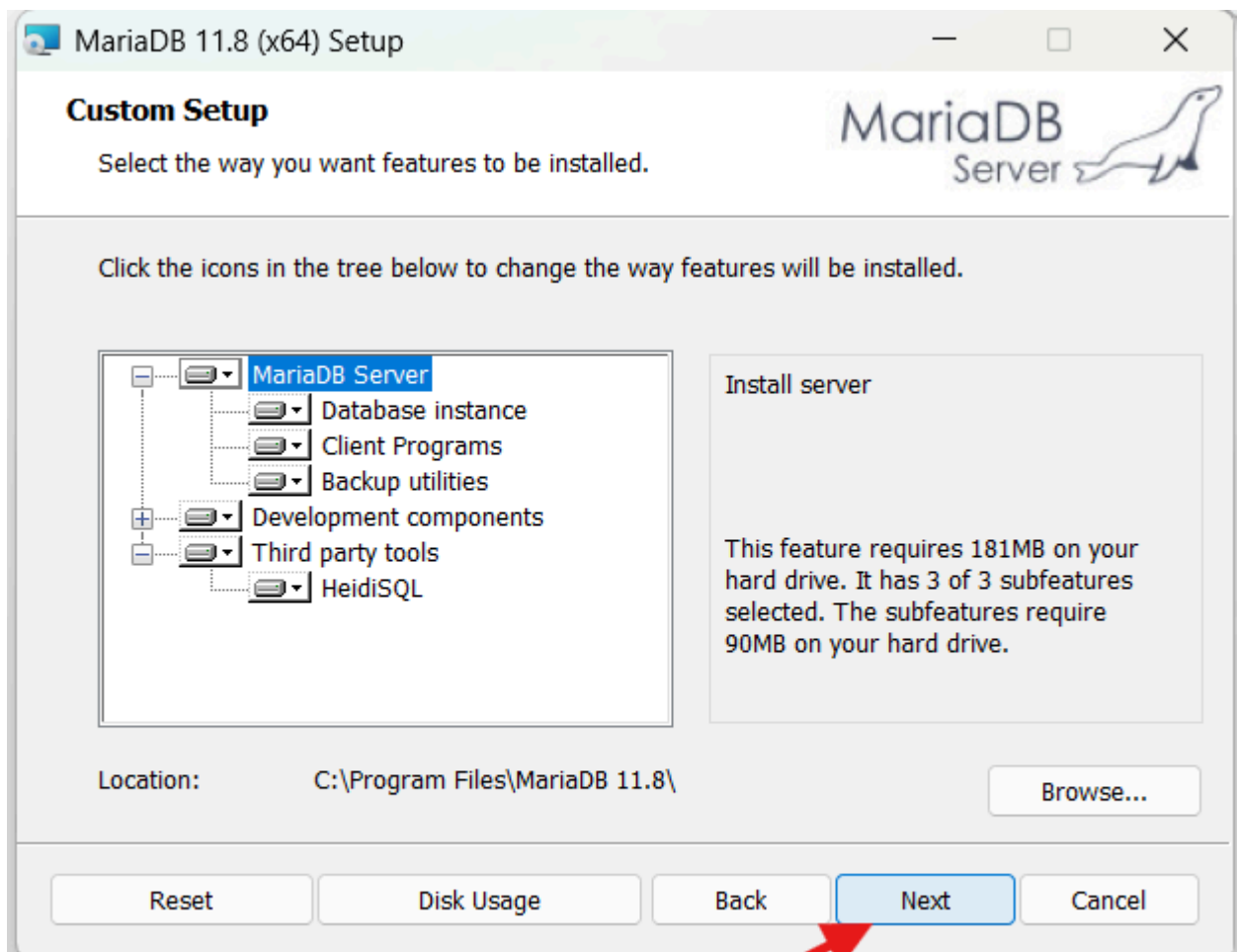


The screenshot shows the MariaDB Server download interface. At the top, there are three tabs: 'MariaDB Server' (selected), 'MariaDB Server Repositories', and 'Connectors'. Below the tabs, the 'MariaDB Server Version' is set to 'MariaDB Server 11.8.2'. There is a checkbox for 'Display older releases' which is unchecked. A link to 'See the MariaDB 11.8.2 Release Notes and Changes and Improvements in MariaDB 11.8.' is provided. The 'Operating System' dropdown is set to 'Windows', and the 'Architecture' dropdown is set to 'x86_64'. The 'Package Type' dropdown is set to 'MSI Package'. A red arrow points to the 'Windows' dropdown. Below these, there is a blue 'Download' button and a 'Mirror' dropdown set to 'Liquid Telecom - Nairobi'. Another red arrow points to the 'Download' button. At the bottom, a box displays release information: 'Release date: 2025-06-05', 'File name: mariadb-11.8.2-winx64.msi', 'File size: 76.8 MB', and a link to 'Download galera-26.4.22/'. A link to 'Display signature and checksums' is also present.

3. Download the MSI installer. It is not necessary to login or sign up after the download.
4. Once it is downloaded, open the installer. Allow MariaDB to make changes to your device.

5. After launching the installer follow the on-screen steps.





6. During installation:

- **When asked to add a password, be sure to make a note of it so you don't forget it!** You can keep root as the user.

User settings

Default instance properties

MariaDB 11.8 (x64) database configuration

MariaDB Server 

☒ **Modify password for database user 'root'**

New root password: Enter new root password

Confirm: Retype the password

☐ **Enable access from remote machines for 'root' user**

☐ **Use UTF8 as default server's character set**

Data directory

- Accept default port and authentication options unless you have specific needs.

Database settings

Default instance properties

MariaDB 11.8 (x64) database configuration

MariaDB Server



☒ **Install as service**

Service Name:

☒ **Enable networking**

TCP port:

InnoDB engine settings

Buffer pool size: MB

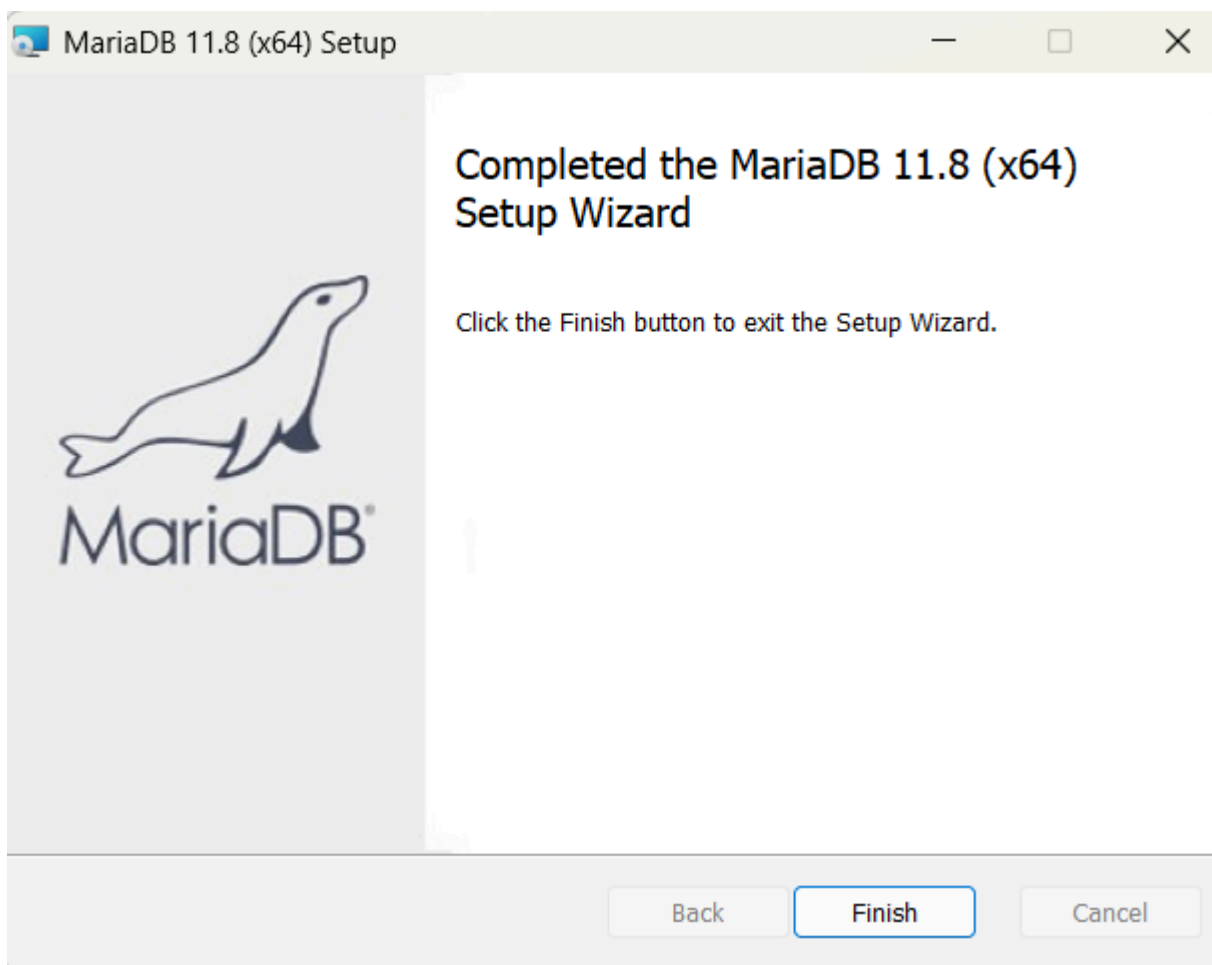
Page size: KB

Back

Next

Cancel

7. Once the installation is complete, MariaDB Server and MariaDB Client will be installed.



For macOS:

1. Download the MariaDB Server for macOS from the official site: <https://mariadb.org/download/>
2. Select **macOS** as your operating system.
3. Download the **macOS PKG installer**. It is not necessary to log in or sign up after the download.
4. Once the download is complete:
 - a. **Double-click the .pkg file** to launch the installer.

- b. Follow the on-screen instructions to install MariaDB. You may need to enter your system password to proceed with the installation.
- c. During installation, MariaDB will be installed to the `/usr/local/mariadb` directory by default.
- d. You may be prompted to create or enter a **root password**. Be sure to write this down so you don't forget it.
- e. By default, MariaDB uses the standard port and authentication method, so you can leave these as-is unless you have a specific reason to change them.

5. After installation:

- a. Open the **Terminal** application. To verify the installation, type:

```
mariadb
```

- b. Press **Enter**. If installed correctly, you should see the MariaDB command-line interface start up.

Once installed, **MariaDB Server** and **MariaDB Client** will be available on your system and ready for use with JDBC or other tools.

For Linux:

1. Check if your distribution includes MariaDB in the native repository:
 - a. Open a terminal
 - b. Run `apt show mariadb-server` (as superuser if necessary):
 - c. Unless `apt show` returns "E: No packages found," skip to "Install MariaDB from native package manager" below.
2. Download the DEB package from the [**MariaDB website**](#):
 - a. Select your operating system and version.
 - b. It is not necessary to log in or sign up after the download.
3. Install MariaDB from downloads:
 - a. Open a terminal in the folder where you saved the DEB package

- b. Run: `sudo dpkg -i [name of DEB package]`
 - c. At some point during installation, you will be prompted to enter a root password. It is critical you remember this! It is difficult to reset.
 - d. Installation is complete.
4. Install MariaDB from native package manager:
 - a. From the terminal, run: `sudo apt-get install mariadb-server`
 - b. At some point during installation, you will be prompted to enter a root password. It is critical you remember this! It is difficult to reset.
 - c. After the server finishes installing, run: `sudo apt-get install mariadb-client`
 - d. Congratulations! Installation is complete.

Using MariaDB

We will now briefly discuss the basics of using MariaDB. Some of the examples below are based on Hock-Chuan of Nanyang Technological University's (2020a) guide to MySQL, adapted here for MariaDB.

Starting the Server

MariaDB is a client-server system. This means that the database runs as a server application, and users can access the database server locally or remotely using a client program. To start up the server you need to do the following:

For Windows:

- If you used the MSI installer, MariaDB should be running once your computer starts.

For Mac:

Use Homebrew to manage the service—most installs are performed via Homebrew.

1. To start and enable automatic startup:
 - a. Open your Terminal and run `brew services start mariadb`

2. For a one-time, manual start (not automatically enabled):
 - a. Open your Terminal and run `mariadb-server start`. If `mariadb-server start` is unavailable, use `mysql.server start`

The MariaDB server should now be started and able to handle requests from the client.

Note:

- The `mysql.server start` command **works** with MariaDB **if the MariaDB Homebrew formula includes it**, but this can vary based on how MariaDB was installed (e.g., `mariadb`, `mariadb@10.6`, etc.).
- As of recent Homebrew versions, **brew services** is the recommended way to start MariaDB persistently.

For Linux:

1. From the terminal, run `sudo systemctl start mariadb`
2. On older systems:
 - a. Run `sudo service mariadb start`
3. This may prompt you for an admin password, after which it will start up.

Shutting Down the Server

For Windows:

- Press Ctrl+C. This initiates a normal shut down. Do not use the windows close button to close the server.

```
You should get the following message from the server console:
[Note] mysqld: Normal shutdown
InnoDB: Starting shutdown...
InnoDB: Shutdown completed; log sequence number ...
[Note] mysqld: Shutdown complete
```

For Mac:

1. Run `brew services stop mariadb`

This cleanly stops the background service managed by Homebrew and is the standard approach on modern macOS systems

For Linux:

1. From the terminal, run `sudo systemctl stop mariadb`
 - a. This may prompt you for an admin password, after which it will shut down.
2. On older systems or those using SysV init, run `sudo service mysql stop`
 - a. This will also shut down MariaDB, as many distributions link the service under the `mysql` alias.

Starting a Client

MariaDB uses a client-server model. Once the server is running, clients can connect either locally (on the same machine) or remotely (from another system on the network). Below are the steps to launch the command-line client as the superuser **root**:

We will now start a command-line client with the superuser **root**:

For Windows:

1. If you installed MariaDB using the **MSI installer**, the binaries are typically located in: `C:\Program Files\MariaDB <version>\bin`.
 - a. For example `C:\Program Files\MariaDB 11.4\bin`
2. Open the Command Prompt and navigate to the MariaDB `bin` directory:
 - a. Run `cd "C:\Program Files\MariaDB 11.4\bin"`
 - b. Adjust the version number as needed.

3. Start the MariaDB client as "root" by running `mariadb -u root -p`
4. When prompted, enter the **root password** (you won't see any characters as you type).
5. If successful, you'll see something like:

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 11.4.0-MariaDB MariaDB Server
```

For Mac:

- Firstly, make sure you know the directory path for the MariaDB bin folder.
- Open a new terminal and change the directory to your MariaDB bin.

```
# Find the MariaDB binary directory
cd /usr/local/mariadb/bin

# Start a client with "root"
./mariadb -u root -p
Enter password:
    # Enter the root's password
    # Nothing will be shown when typing the password

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 11.4.0-MariaDB MariaDB Server

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.

mariadb>
# You can now enter SQL commands.
```

For Linux:

- From the terminal, run `mariadb -u root -p`
- If access is denied even if the password is correct, it may be necessary to run this as a superuser by using `sudo mariadb -u root -p`.

Changing the Password for Root

The superuser "root" can do anything to the databases. This includes deleting all of them. Therefore, security is imperative when it comes to the "root" superuser. This means that you need to change the root's temporary password immediately after logging in.

Your client should now be started. We will, therefore, continue with the current client session. Simply enter the following:

```
// Replace XXXXX with your chosen password, enclosing it in single quotes
because it is a string
ALTER USER 'root'@'localhost' IDENTIFIED BY 'XXXXX';
```

If the command succeeds, you'll see:

```
Query OK, 0 rows affected (0.00 sec)
```

Then exit the client by running `QUIT;`
:

Create a New User

The superuser "root" is a privileged user and is meant to be used for database administration. It is not meant to be used for everyday operational purposes. We, therefore, should create another user with fewer privileges. We will call this user "otheruser". To create a new user you need to start a client with the "root" superuser as shown above.

```

# If not already started, open the MariaDB client as root

# On Windows:
mariadb -u root -p

# On macOS:
./mariadb -u root -p

// Create a new user called 'otheruser' with password 'swordfish'
CREATE USER 'otheruser'@'localhost' IDENTIFIED BY 'swordfish';

// The new user has no privileges yet

// Grant all privileges on all databases and tables to 'otheruser'
GRANT ALL ON *.* TO 'otheruser'@'localhost';

// Exit the client
QUIT;

```

This grants **otheruser** full privileges on all databases, similar to **root** except without the ability to grant privileges to others. For everyday use, consider granting more limited privileges instead for better security.

Client Session Tips

Before we go any further, here are some tips on using the client:

- Terminate your command with a semicolon (;) like in Java.
- A single command can span many lines. To show continuation after pressing Enter, the new line prompt changes to ->. Once you are finished with the command add the ; at the end.
- Use \c to cancel (abort) the current command
- If you open a single quote without closing it the new line prompt changes to '> instead of ->
- Use the up and down arrow keys to get previous or next commands from the command history.

Creating a New Database and a New Table, Inserting Records, Querying and Updating

A MariaDB server contains many databases. In turn, a database contains many tables and a table contains many rows (records) and columns (fields).

We will now create a database called “dogs_db” and a table called “java_programming”. The table shall have three columns: id (of the type int), name (of the type varchar(50)) and weight (of the type float).

```
# Start a client with the new user "otheruser"
# cd to the MariaDB bin directory if needed

# Windows:
mariadb -u otheruser -p

# macOS:
./mariadb -u otheruser -p
-- Create a new database called 'dogs_db' if it doesn't already exist
CREATE DATABASE IF NOT EXISTS dogs_db;

-- List all the databases on this server
SHOW DATABASES;
+-----+
| Database |
+-----+
| ..... |
| dogs_db |
| ..... |
+-----+
-- x rows in set (0.07 sec)

-- Use "dogs_db" as the default database
USE dogs_db;
Database changed

-- Remove the table "letters" if it exists in the current database
DROP TABLE IF EXISTS letters;
Query OK, 0 rows affected, 1 warning (0.15 sec)
```

```
-- Create a new table called "java_programming" with 3 columns: id, name, and weight
CREATE TABLE java_programming (
  id INT,
  name VARCHAR(50),
  weight FLOAT
);
```

Running SQL Commands from a File in MariaDB

Instead of typing commands one by one, you can put multiple SQL statements into a file and execute them all at once.

1. Using a text editor, create a file named `mycommands.sql` containing your SQL statements, for example:

```
CREATE DATABASE IF NOT EXISTS dogs_db;
USE dogs_db;
CREATE TABLE java_programming (
  id INT,
  name VARCHAR(50),
  weight FLOAT
);
INSERT INTO java_programming (id, name, weight) VALUES (1, 'Fluffy', 12.2);
INSERT INTO java_programming (id, name, weight) VALUES (3, 'Patch', 6.0);
INSERT INTO java_programming (id, name, weight) VALUES (4, 'Denzel', 54.0);

SELECT * FROM java_programming;
```

- a. Save the file under `d:\myProject\mycommands.sql` on Windows and under `~/Documents/mycommands.sql` on Mac OS
2. After you created the file, you can use the following source command to run the SQL script:

```
// For Windows ONLY
mariadb> source d:\myProject\mycommands.sql;
```

```
// For macOS ONLY
mariadb> source ~/Documents/mycommands.sql;
```

Expected output:

```
Query OK, 1 row affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
Query OK, 1 row affected (0.00 sec)
```

```
+-----+-----+-----+
| id    | name   | weight |
+-----+-----+-----+
| 1     | Fluffy | 12.2   |
| 3     | Patch | 6.0    |
| 4     | Denzel | 54.0   |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

This was a short introduction to SQL. Please use the [Reference Manual](#) to learn more.

Developing Database Applications using JDBC

According to Liang (2011, p. 1287), “The JDBC API consists of classes and interfaces for establishing connections with databases, sending SQL statements to databases, and processing the results of SQL statements.” This means that a program written in Java can access **any relational database, including MariaDB**, by using SQL.

In general, a JDBC program connecting to MariaDB involves the following steps:

1. **Connect to a MariaDB server** by allocating a `Connection` object.
Typically, you’ll use the MariaDB JDBC driver (`org.mariadb.jdbc.Driver`) and a connection URL like:
`jdbc:mariadb://hostname:port/databaseName`
2. **Allocate a Statement object**, created from the `Connection`, for holding a SQL command.
3. **Write and execute SQL queries** using the `Statement` and `Connection` objects.

4. **Process the results** returned from the query, usually via a `ResultSet` object.
5. **Close the Statement and Connection objects** to free resources when finished.

Common JDBC Components

- **DriverManager:**
A class that manages the list of database drivers. The `DriverManager` matches connection requests from the Java application with the correct database driver based on the connection URL's subprotocol (e.g., `jdbc:mariadb:`). The first driver that recognizes the subprotocol is used to establish the database connection.
- **Driver:**
An interface that handles the low-level communication with the database server. For MariaDB, you typically use the MariaDB JDBC driver (`org.mariadb.jdbc.Driver`). Usually, you don't interact directly with `Driver` objects — instead, you work through `DriverManager`, which abstracts driver management.
- **Connection:**
An interface representing a connection session with the database. All communication with the MariaDB server is done through a `Connection` object.
- **Statement:**
An interface used to submit SQL statements (queries, updates, etc.) to the MariaDB database.
- **ResultSet:**
Objects returned by executing SQL queries via a `Statement`. They hold the data retrieved from MariaDB and provide methods to iterate through the rows.
- **SQLException:**
A class representing exceptions and errors that occur while interacting with the MariaDB database through JDBC.

Installing MariaDB JDBC Driver

To run your Java database programs with MariaDB, you need to install the appropriate JDBC driver. The MariaDB JDBC driver, called **MariaDB Connector/J**, is available for download from the official MariaDB website. Unlike MySQL's Connector/J, it is **not bundled automatically** with MariaDB installers, so you need to download and add it manually to your project.

MariaDB Connector/J is a fully compatible JDBC driver designed specifically for MariaDB databases.

The information below is adapted from Hock-Chuan's explanations and guides (2020b), tailored for MariaDB.

To install the MariaDB JDBC Connector on Windows:

If you installed MariaDB using the MSI installer, the **MariaDB Connector/J** may already be included. If not, follow the steps below to install it manually:

1. Visit the official MariaDB website to download the latest **MariaDB Connector/J** (JDBC driver).
2. Under **Connectors**, choose **MariaDB Connector/J**.
3. Select the **Platform Independent** version, typically available as a **.zip archive**.
4. Click **"No thanks, just start my download"** (or similar link if prompted to log in).
5. Once downloaded, **unzip** the archive into a temporary folder.
6. Locate the **.jar** file inside the unzipped folder — this is the actual connector.
7. Copy the **.jar** file to your **Java Extension Directory**:
C:\Program Files\Java\jdk1.8.0_{xx}\jre\lib\ext
(Replace {xx} with your actual JDK update version)

Note: If you are using a newer version of Java where the **ext** directory is deprecated, you may instead include the **.jar** in your project's classpath manually through your IDE or build tool (like Maven or Gradle).

To Install the MariaDB JDBC Connector on macOS:

1. Visit the official MariaDB website to download the latest **MariaDB Connector/J**.
2. Under the **Connectors** section, choose **MariaDB Connector/J**.
3. Select the **Platform Independent** option — typically provided as a **compressed TAR archive**.
4. Click **"No thanks, just start my download"** to begin downloading.

5. Once the download is complete, **double-click the .tar.gz file** to extract its contents.
6. Inside the extracted folder, locate the .jar file — this is the JDBC connector.
7. Copy the .jar file into your system-wide Java Extension Directory:
/Library/Java/Extensions

Tip: On newer versions of macOS or Java, if the Extensions folder doesn't exist or isn't supported, you can instead reference the .jar directly in your project or add it to your classpath using your development environment.

To Connect Your Java Application to Your MariaDB Server in Eclipse:

1. Open **Eclipse** and create a **new Java project**.
2. When prompted, instead of clicking **Finish** immediately, click **Next** to continue with additional project configuration.
3. In the next screen, click on the **“Libraries”** tab (located near the top of the window).
4. Under the Libraries section, choose **“Classpath”**.
5. Click **“Add External JARs...”**.
6. Navigate to the location where you saved the **MariaDB JDBC connector .jar file** (e.g., mariadb-java-client-x.x.x.jar) and select it.
7. Click **Open** to add the connector to your project, and then **Finish** to complete the setup.

This step ensures your Java application can communicate with the MariaDB server using JDBC.

Setting up a Database

Before we can go any further, we need to set up a database. We will call this database “library_db” and create a table called “books” within it with 4 columns: id, title, author, and qty.

id	title	author	qty
(int)	(varchar(50))	(varchar(50))	(int)

1001	Java for Beginners	John Holder	5
1002	Java Fundamentals	Sally Williams	5
1003	A Cup of Java	Peter Jones	6
1004	Introduction to Java	Kumar Singh	6
1005	Advanced Java	Kelly Fields	7

- As shown previously, to create a database you need to start the MariaDB server and start the MariaDB client:

For Windows:

1. Open **Command Prompt**.
2. Navigate to the MariaDB **bin** directory. For example:

```
cd {path-to-mariadb-bin} // Check your MariaDB installed directory
mariadb -u otheruser -p
```

For macOS:

1. Start the MariaDB server from:
 - **Apple Icon** → **System Preferences** → **MariaDB** → **Start**
2. Open **Terminal**.
3. Navigate to the MariaDB installation folder:

```
cd /usr/local/mariadb/bin
./mariadb -u otheruser -p
```

Note: remember to enter the password for the user “otheruser” and not “root”

- Now, run the following SQL statements to create the database and table. Not that this time you will set up the table to use the ID field as the index, i.e. primary key.

```
create database if not exists library_db;

use library_db;

drop table if exists books;
create table books (
  id int,
  title varchar(50),
```

```
author varchar(50),
qty int,
primary key (id));
insert into books values (1001, 'Java for Beginners', 'John Holder', 5);
```

JDBC Programming

We will now demonstrate JDBC programming using some examples (Hock-Chuan, 2020b). The code below shows how to select, insert, update and delete entries in a database.

Issuing SQL statements from Java:

- Create a new Java source file called `Main.java`. (Remember: the file name must be the same as the class name)
- Save the file in a directory of your choice.
- Enter the following code into your newly created file:

```
import java.sql.*;

public class Main {

    public static void main(String[] args) {
        try {
            // Connect to the library_db database via the jdbc:mariadb:
channel on localhost
            // Use username "otheruser" and password "swordfish".
            Connection connection = DriverManager.getConnection(
                "jdbc:mariadb://localhost:3306/library_db",
                "otheruser",
                "swordfish"
            );

            // Create a direct line to the database for running queries
            Statement statement = connection.createStatement();
            ResultSet results;
            int rowsAffected;

            // SELECT: Retrieve book titles and quantities
```

```

        results = statement.executeQuery("SELECT title, qty FROM
books");
        while (results.next()) {
            System.out.println(results.getString("title") + ", " +
results.getInt("qty"));
        }

        // INSERT: Add a new book
        rowsAffected = statement.executeUpdate(
            "INSERT INTO books VALUES (3001, 'Programming 101',
'Jane Doe', 1)"
        );
        System.out.println("Query complete, " + rowsAffected + "
rows added.");
        printAllFromTable(statement);

        // UPDATE: Change the quantity of an existing book
        rowsAffected = statement.executeUpdate(
            "UPDATE books SET qty=500 WHERE id=1001"
        );
        System.out.println("Query complete, " + rowsAffected + "
rows updated.");
        printAllFromTable(statement);

        // DELETE: Remove a book from the table
        rowsAffected = statement.executeUpdate(
            "DELETE FROM books WHERE id=3001"
        );
        System.out.println("Query complete, " + rowsAffected + "
rows removed.");
        printAllFromTable(statement);

        // Close all connections
        results.close();
        statement.close();
        connection.close();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

/**
 * Prints all rows from the 'books' table.
 *
 * @param statement A SQL statement on an existing connection
 * @throws SQLException if a database access error occurs
 */
public static void printAllFromTable(Statement statement) throws
SQLException {
    ResultSet results = statement.executeQuery("SELECT id, title,
author, qty FROM books");
    while (results.next()) {
        System.out.println(
            results.getInt("id") + ", "
            + results.getString("title") + ", "
            + results.getString("author") + ", "
            + results.getInt("qty")
        );
    }
}
}

```



Extra resource

You may encounter an error when trying to run your code that says: "SQLException: Access denied for user 'otheruser'@'localhost' ". This usually means that the database is rejecting the login because of how the user is authenticated. In MariaDB, you can fix this by logging into the MariaDB client using a privileged account like root, and then updating the authentication method for the user. Use the following command: `ALTER USER 'otheruser'@'localhost' IDENTIFIED VIA mysql_native_password USING PASSWORD('swordfish');` — replacing 'swordfish' with your actual password. After that, run `FLUSH PRIVILEGES;` to apply the changes. This tells MariaDB to use a compatible authentication plugin that works with JDBC. Once you've made this update, your Java application should be able to connect to the database successfully.

Let's take a look at the program you just wrote. Firstly, notice that there is not much actual programming involved in JDBC programming. You simply need to specify the database-URL, write the SQL query, and process the query result. The rest of the code can be thought of as a standard template for a JDBC program.

Now let's look at the following lines of code:

```
Connection connection = DriverManager.getConnection(
    "jdbc:mariadb://localhost:3306/library_db",
    "otheruser",
    "swordfish"
);
```

`jdbc:mariadb://localhost:3306/library_db` is the database-URL and takes the following form: `jdbc:mariadb://{host}:{port}/{database-name}`

Port represents the TCP port number of the MariaDB server, in most cases, this is port **3306** by default and database-name is the name of the database, which is in this instance `library_db`. "otheruser" and "swordfish" are the username and password of an authorised MariaDB user.

Next let's examine this statement:

```
Statement statement = connection.createStatement();
```

In this line, we are allocating a `Statement` object inside the `Connection` using `connection.createStatement()`.

Select

The next step in our process is to execute a SQL command (in this case `Select`). To do this we use the method `statement.executeQuery("SELECT ...")`, which returns the query result in a `ResultSet` object called `results`. The `ResultSet` models the table which is returned. This table can then be accessed using a row cursor. The cursor is initially positioned before the first row in `ResultSet`. It is then moved, using `results.next()`, to the first row. `results.getXxx(column Name)` can then be used to retrieve the value of the column for that row. `Xxx` corresponds to the data type of that column, for example, `int`, `float`, `double` and `String`. At the last row the `results.next()` returns `false`, which terminates the while-loop. `results.getString(columnName)` can also be used to retrieve all data types.

ResultSet columns within each row should be read in a left-to-right order, and each column should be read only once using the `getXxx()` methods. Issuing `getXxx()` to a cell more than once can cause an error.

A feature of JDK called try-with-resources automatically closes all the opened resources in the try-clause. It therefore closes the Connection and Statement objects automatically.

Insert and Delete

The insert and delete actions are fairly straightforward, as you can see from the code above. It is worth noting that you cannot add an entry if an existing entry has the same primary key; you will first have to delete the existing entry.

Update

Unlike Select, where we use `executeQuery()` to return a ResultSet object, Update, Insert and Delete return an int value, which indicates the number of records affected..

For more information on JDBC, you can visit the [homepage](#) or the [JDBC online tutorial](#).

Testing and Debugging

Normally the first step in debugging is to attempt to reproduce the problem. This can be a non-trivial task, for example as with parallel processes or some unusual software bugs. Also, specific user environments and usage history can make it difficult to reproduce the problem.

After the bug is reproduced, the input of the program may need to be simplified to make it easier to debug. For example, a bug in a compiler can make it crash when parsing some large source file. However, after simplification of the test case, only a few lines from the original source file can be sufficient to reproduce the same crash. Such simplification can be made manually, using a divide-and-conquer approach. The programmer will try to remove some parts of the original test case and check if the problem still exists. When debugging the problem in a GUI, the programmer can try to skip some user interaction from the original problem description and check if remaining actions are sufficient for bugs to appear.

After the test case is sufficiently simplified, a programmer can use a debugger tool to examine program states (values of variables, plus the call stack) and track down the origin of the problem(s). Alternatively, tracing can be used. In simple cases, tracing is just a few print statements, which output the values of variables at certain points of program execution.

- **Print debugging** (or tracing) is the act of watching (live or recorded) trace statements, or print statements, that indicate the flow of execution of a process.
- **Remote debugging** is the process of debugging a program running on a system different from the debugger. To start remote debugging, a debugger connects to a remote system over a network. The debugger can then control the execution of the program on the remote system and retrieve information about its state.
- **Post-mortem debugging** is debugging of the program after it has already crashed. Related techniques often include various tracing techniques, and/or analysis of memory dump (or core dump) of the crashed process. The dump of the process could be obtained automatically by the system (for example, when the process has terminated due to an unhandled exception), or by a programmer-inserted instruction, or manually by the interactive user.
- **"Wolf fence" algorithm:** Edward Gauss described this simple but very useful and now famous algorithm in a 1982 article for communications of the ACM as follows: "There's one wolf in Alaska; how do you find it? First build a fence down the middle of the state, wait for the wolf to howl, determine which side of the fence it is on. Repeat the process on that side only, until you get to the point where you can see the wolf." This is implemented e.g. in the Git version control system as the command `git bisect`, which uses the above algorithm to determine which commit introduced a particular bug.
- **Delta Debugging** - technique of automating test case simplification
- **Saff Squeeze** - technique of isolating failure within the test using progressive inlining of parts of the failing test.

Documentation

The documents associated with a software project and the system being developed have a number of associated requirements:

1. They should act as a communication medium between members of the development team.
2. They should be a system information repository to be used by maintenance engineers.
3. They should provide information for management to help them plan, budget, and schedule the software development process.
4. Some of the documents should tell users how to use and administer the system.

Product documentation

Product documentation is concerned with describing the delivered software product. Unlike most process documentation, it has a relatively long life. It must evolve in step with the product which it describes. Product documentation includes user documentation which tells users how to use the software product and system documentation which is principally intended for maintenance engineers.

User Documentation

Users of a system are not all the same. The producer of documentation must structure it to cater for different user tasks and different levels of expertise and experience. It is particularly important to distinguish between end-users and system administrators:

1. End-users use the software to assist with some task. This may be flying an aircraft, managing insurance policies, writing a book, etc. They want to know how the software can help them.
2. System administrators are responsible for managing the software used by end-users. This may involve acting as an operator if the system is a large mainframe system, as a network manager in the system involves a network of workstations or as a technical guru who fixes end-users software problems and who liaises between users and the software supplier.

To cater for these different classes of user and different levels of user expertise, there are at least 5 documents (or perhaps chapters in a single document) which should be delivered with the software system.

The functional description of the system outlines the system requirements and briefly describes the services provided. This document should provide an overview of the system. Users should be able to read this document with an introductory manual and decide if the system is what they need.

The system installation document is intended for system administrators. It should provide details of how to install the system in a particular environment. It should contain a description of the files making up the system and the minimal hardware configuration required. The permanent files which must be established, how to start the system and the configuration dependent files which must be changed to tailor the system to a particular host system should also be described. The use of automated installers for PC software has meant that some suppliers see this document as unnecessary. In fact, it is still required to help system managers discover and fix problems with the installation.

The introductory manual should present an informal introduction to the system, describing its 'normal' usage. It should describe how to get started and how end-users

might make use of the common system facilities. It should be liberally illustrated with examples. Inevitably beginners, whatever their background and experience, will make mistakes. Easily discovered information on how to recover from these mistakes and restart useful work should be an integral part of this document.

A more general system administrator's guide should be provided for some types of system such as command and control systems. This should describe the messages generated when the system interacts with other systems and how to react to these messages. If system hardware is involved, it might also explain the operator's task in maintaining that hardware. For example, it might describe how to clear faults in the system console, how to connect new peripherals, etc.

As well as manuals, other, easy-to-use documentation might be provided. A quick reference card listing available system facilities and how to use them is particularly convenient for experienced system users. On-line help systems, which contain brief information about the system, can save the user spending time in consultation of manuals although should not be seen as a replacement for more comprehensive documentation.

System Documentation

System documentation includes all of the documents describing the system itself from the requirements specification to the final acceptance test plan. Documents describing the design, implementation and testing of a system are essential if the program is to be understood and maintained. Like user documentation, it is important that system documentation is structured, with overviews leading the reader into more formal and detailed descriptions of each aspect of the system.

For large systems that are developed to a customer's specification, the system documentation should include:

1. The requirements document and an associated rationale.
2. A document describing the system architecture.
3. For each program in the system, a description of the architecture of that program.
4. For each component in the system, a description of its functionality and interfaces.
5. Program source code listings. These should be commented where the comments should explain complex sections of code and provide a rationale for the coding method used. If meaningful names are used and a good, structured programming style is used, much of the code should be self-documenting without the need for additional comments. This information is now normally maintained electronically rather than on paper with selected information printed on demand from readers.

6. Validation documents describing how each program is validated and how the validation information relates to the requirements.



Practical task 1

Follow these steps:

1. Ensure that your environment is set up and you have followed all the steps outlined in this task.
2. Using the MariaDB client:
 - create and use a **database** called `java_db`
 - create a **table** called `students` with the fields (`ID int`, `Name varchar(50)`, `Grade float`)
3. Now, follow the instructions below. After executing each instruction, take a **screenshot** of your console showing the query you wrote and the results, and add it to a **Google doc** which you will submit as part of your answer. Number your screenshots 1 to 5 in order of execution.
 - Insert the following 3 new rows of data into the students table:

ID	Name	Grade
55	Carl Davis	61
66	Dennis Fredrickson	88
77	Jane Richards	78

- Select all records with a grade between 60 and 80.
- Change Carl Davis's grade to 65.
- Delete Dennis Fredrickson's row.

- Change the grade of all people with an ID greater than 55 to 80.
4. Ensure that you've taken all the required screenshots and pasted them into your Google doc for later submission.

Now, you'll be working on adding some functionality to some existing programs that have been provided for you. Open the Java program `main.java`. Modify it according to the instructions below:

- Set the qty for Introduction to Java to 0.
- Delete all books with id > 8000
- Insert:
 - 8001, 'Java ABC', 'Kevin Jones', 3, and
 - 8002, 'Java XYZ', 'Kevin Jones', 5
- Test and debug your code
- Submit your modified program, as well as your Google document from the previous question.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Practical task 2

1. Create a Java program that can be used by a bookstore clerk. Use the comment functionality to document what you have done making the code easily understandable to anyone reading your program. The program should allow the clerk to:
 - enter new books into the database
 - update book information
 - delete books from the database

- search the database to find a specific book.

2. Create a database called **ebookstore** and a table called **books**. The table should have the following structure (note that the ID field is the primary key):

ID	Title	Author	Qty
3001	A Tale of Two Cities	Charles Dickens	30
3002	Harry Potter and the Philosopher's Stone	J.K. Rowling	40
3003	The Lion, the Witch and the Wardrobe	C. S. Lewis	25
3004	The Lord of the Rings	J.R.R Tolkien	37
3005	Alice in Wonderland	Lewis Carroll	12

3. Populate the table with the above values. You can also add your own values if you wish.
4. Now, write your Java program to interact with the database. The program should present the user with the following menu:
 1. Enter book
 2. Update book
 3. Delete book
 4. Search books
 0. ExitThe program should perform the function that the user selects. The implementation of these functions is left up to you.
5. Include comments to explain complex sections of code and provide a rationale for the coding method used.
6. Test and debug your code.
7. Feel free to add more functionality and complexity to the program. This is your chance to show off all the programming concepts you have learnt so far!

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback [form](#) to help us ensure we provide you with the best possible learning experience.

Reference list

Ciubotaru, B., & Muntean, G. (2013). Advanced network programming – Principles and techniques: Network application programming with Java (p. 136). London: Springer Science & Business Media.

Hock-Chuan, C. (2020a). Java Tutorial - An Introduction to Java Database Programming (JDBC) by Examples with MySQL. Retrieved 19 February 2020, from https://www3.ntu.edu.sg/HOME/EHCHUA/PROGRAMMING/java/JDBC_Basic.html

Hock-Chuan, C. (2020b). An Introduction to Java Database Programming (JDBC) by Examples. Retrieved 20 February 2020, from https://www3.ntu.edu.sg/HOME/EHCHUA/PROGRAMMING/java/JDBC_Basic2.html

Liang, Y. (2011). Introduction to Java programming: Comprehensive version (8th ed., pp. 1273-1308). New Jersey: Prentice Hall.

